



INSTITUTO FEDERAL DO NORTE DE MINAS GERAIS
CAMPUS MONTES CLAROS -MG
CIÊNCIA DA COMPUTAÇÃO



SARAH EMANUELLE ALVES LINO
TAINARA RIBEIRO SANTOS

RELATÓRIO: PRÁTICA DE LABORATÓRIO 7

Relatório apresentado ao Professor Wagner Ferreira de Barros como parte das exigências de avaliação da disciplina de Organização e Sistemas de Arquivos do Curso de Bacharelado em Ciência da Computação do Instituto Federal do Norte de Minas Gerais, Campus Montes Claros.

MONTES CLAROS - MG

2025

Relatório do Projeto: Implementação de uma Árvore B

1. Introdução

A Árvore B é uma estrutura de dados amplamente utilizada em sistemas de gerenciamento de banco de dados e em sistemas de arquivos devido à sua capacidade de manter dados ordenados e permitir buscas, inserções e remoções eficientes.

Nesse sentido, o objetivo principal desse trabalho foi a implementação de uma Árvore B em C++, com a adição de funcionalidades para armazenar a estrutura em arquivos binários e texto, possibilitando a visualização e manipulação da árvore entre execuções do programa. A implementação também buscou explorar operações fundamentais da árvore B, como inserção, remoção e busca, com a adição de testes automatizados para garantir a robustez do código.

Além disso, é importante ressaltar que a Árvore B escolhida para o desenvolvimento do presente trabalho seguiu a definição original desta estrutura de dados dada por Bayer e McCreight em 1970, onde a ordem da árvore é definida pelo número mínimo de chaves por nó (ou página) e não pelo número de filhos, conforme especificado na requisição do trabalho.

2. Metodologia

2.1. Estrutura do Projeto

O projeto foi estruturado de forma modular, onde cada parte do código é responsável por uma funcionalidade específica, conforme explicitado a seguir:

2.1.1 Arquivos Fonte

- *BTree.h*: Cabeçalho que define a estrutura da Árvore B, incluindo as operações que podem ser realizadas, como inserção, remoção, busca, impressão e salvamento da árvore em arquivo. Também são definidas as estruturas internas necessárias para a árvore, como nós e os ponteiros para os filhos e pais dos nós. As funções auxiliares como a divisão e fusão de nós também são declaradas neste arquivo.

- *BTree.cpp*: Arquivo responsável pela implementação das operações declaradas no cabeçalho da *BTree.h*. Este arquivo contém as funções de manipulação da árvore, como a inserção e remoção de nós, divisão de nós e busca dentro da árvore.
- *main.cpp*: Arquivo principal que contém a lógica de execução do programa, realizando inserções, remoções, buscas e exibindo o estado da árvore. Além disso, este arquivo realiza o armazenamento da árvore em arquivos binários e de texto para garantir que a árvore esteja disponível entre as execuções do programa. Ele também gerencia a exibição do estado atual da árvore.

2.1.2 Makefile

- *Makefile*: foi criado para facilitar a compilação e execução do programa. Ele inclui a definição de flags para a compilação, como opções de depuração e otimização, além de automatizar a criação dos arquivos objeto e do executável. O *Makefile* também inclui comandos para limpar os arquivos temporários gerados durante a compilação, garantindo que o processo de construção do programa seja sempre feito de maneira limpa e eficiente.

2.1.3 Arquivos de Armazenamento

- *btree.bin*: Arquivo que armazena a estrutura da Árvore B em formato binário. Esse arquivo é utilizado para salvar os dados da árvore, incluindo as chaves, endereços dos filhos e pais e outras informações associadas aos nós. Ele permite que a árvore seja restaurada de forma rápida e eficiente em execuções subsequentes do programa.
- *btree.txt*: Arquivo de texto gerado a partir da estrutura binária da árvore. Ele contém uma versão legível da árvore, incluindo informações sobre os nós, como chaves, endereços dos filhos e pais, e se o nó é uma folha. Esse arquivo é útil para visualizar a estrutura da árvore de forma manual e inspecionar o seu estado em uma forma mais acessível.

2.2 Funcionalidades

2.2.1 Inserção de Chaves

A inserção na Árvore B segue o princípio de manter a ordem dos nós enquanto balanceia a árvore para evitar que ela se torne desbalanceada. A operação de inserção funciona da seguinte forma:

- Localização do nó correto: O código começa localizando o nó correto onde a chave será inserida, comparando a chave com as chaves existentes no nó.
- Inserção em nó não cheio: Se o nó tiver espaço para uma nova chave (ou seja, se o número de chaves no nó for inferior a $2 \times \text{ordem}$), a chave é simplesmente inserida na posição apropriada, mantendo as chaves ordenadas.
- Divisão de nó: Se o nó estiver cheio, o nó é dividido. A chave do meio é promovida para o nó pai, e os dois nós resultantes (antes e depois da chave do meio) são redistribuídos. Se o nó pai também estiver cheio, o processo de divisão é recursivo. Esse processo garante que a Árvore B permaneça balanceada, com a altura da árvore limitada.
- Aumento da altura da árvore: Se a divisão alcançar a raiz e a raiz estiver cheia, a árvore aumentará em altura, com uma nova raiz sendo criada. A ordem da árvore (mínimo de chaves por nó) é respeitada em todo o processo.

2.2.2 Remoção de Chaves

A remoção de chaves na Árvore B também segue as regras para garantir a árvore balanceada após a exclusão de um elemento:

- Remoção de chave de nó folha: Se a chave a ser removida estiver em um nó folha, o nó simplesmente remove a chave. Se a quantidade de chaves no nó cair abaixo do número mínimo (definida pela ordem), o nó será balanceado por fusão ou empréstimo de um irmão.
- Remoção de chave de nó interno: Se a chave a ser removida estiver em um nó interno, o código encontra o sucessor ou predecessor da chave, substitui a chave a ser removida e remove o sucessor/predecessor (que estará em um nó folha). Caso o nó interno se torne subutilizado após a remoção, ele pode ser balanceado por fusão ou empréstimo.
- Empréstimo e fusão: Quando a remoção de uma chave faz com que um nó tenha menos do que o número mínimo de chaves, o nó pode pegar uma chave emprestada de

um irmão, ou se isso não for possível, o nó será fundido com um irmão. Essa fusão garante que os nós irmãos continuem balanceados e respeitem as regras da árvore.

2.2.3 Busca de Chaves

A operação de busca na Árvore B é feita de maneira eficiente devido à estrutura ordenada e balanceada da árvore. A busca funciona da seguinte forma:

- Descida recursiva: A busca começa na raiz da árvore e desce recursivamente até encontrar a chave desejada ou até chegar a um nó folha. Em cada nó, a chave buscada é comparada com as chaves armazenadas no nó.
- Comparação com as chaves do nó: A busca é feita comparando a chave procurada com as chaves armazenadas no nó. Se a chave procurada for menor que a chave no nó, a busca segue para o filho à esquerda, e se for maior, a busca vai para o filho à direita.
- Eficiência da busca: A busca é eficiente, pois, devido ao balanceamento da árvore, o número de comparações em cada nó é reduzido, e a altura da árvore é logarítmica em relação ao número total de elementos.

2.2.4 Impressão da Árvore

A impressão da árvore no console é utilizada para visualizar a estrutura interna da Árvore B, especialmente após inserções ou remoções, facilitando o entendimento do estado atual da árvore. A operação de impressão funciona da seguinte maneira:

- Estrutura de árvore balanceada: A árvore é impressa em formato hierárquico, onde a raiz e os nós internos estão à esquerda e os nós folhas estão à direita.
- Indentação para visualização: A estrutura é impressa com indentação, onde cada nível de indentação representa um nível na árvore.
- Facilidade para inspeção: A impressão é útil para verificar o estado da árvore, especialmente após operações de inserção ou remoção.

2.2.5 Leitura e Salvamento da Árvore em Arquivo

O projeto implementa a funcionalidade de salvar a árvore em arquivos binário e de texto, permitindo que os dados da árvore sejam preservados entre execuções do programa:

- Arquivo Binário: A árvore é salva em um arquivo binário (btree.bin), onde toda a estrutura da árvore, incluindo os nós e suas relações (pais e filhos), é registrada de forma compacta. Esse formato permite a leitura eficiente e a modificação da árvore.
- Arquivo de Texto: A árvore também pode ser convertida para um arquivo de texto (btree.txt). Esse formato fornece uma visão legível da árvore, exibindo as chaves, endereços dos nós e as relações hierárquicas entre os nós.
- Carregamento e recuperação: Ao iniciar o programa, a árvore pode ser carregada a partir do arquivo binário, permitindo a continuidade das operações sem perder os dados.

2.2.6 Busca Direta no Arquivo Binário

Além da busca na árvore em memória, o programa implementa a busca diretamente no arquivo binário. Isso é possível devido à estrutura da árvore ser armazenada no formato binário, a partir da função *searchInFile*, onde a busca pode ser realizada acessando diretamente os endereços dos nós armazenados no arquivo. Esse recurso permite realizar buscas sem carregar toda a árvore em memória, tornando a busca mais eficiente em cenários com grandes volumes de dados.

3. Testes Realizados e Resultados

Para garantir a correção e a eficiência das operações na Árvore B implementada, foram realizados testes, que serão descritos a seguir, juntamente com os resultados obtidos.

3.1 Definição da Ordem da Árvore

A ordem da Árvore B foi definida como 2 para a demonstração dos testes apresentados no presente relatório. Apesar disso, outras ordens para a árvore são aceitas. Tendo definida a ordem 2, isso significa que cada nó da árvore pode armazenar no máximo 3 chaves e no mínimo 2 chaves (considerando a definição de uma Árvore B onde a ordem m define o número mínimo de chaves por nó). Isso pode ser observado no seguinte trecho do código:

```
13 // Define a ordem da árvore
14 BTree btree(2); // Ordem 2, mas pode ser alterado para arvore de ordem 1,3,...,n
```

3.2 Testes de Inserção

A operação de inserção foi testada para verificar se a árvore mantinha o balanceamento após a inserção de chaves. Para isso, foi utilizado um vetor de chaves, definido na função *main*, que pode ser observado a seguir:

```
16 // Vetor de elementos a serem inseridos na árvore
17 vector<int> elementos = {20, 40, 10, 30, 15, 35, 7, 26, 18, 22, 5, 42, 13, 46, 27, 8, 32, 38, 24, 45, 25};
```

Após cada inserção, o estado da árvore é impresso no terminal, onde o resultado final da árvore foi:

```
-----
- Inse  25
[25]
  [10, 20]
    [5, 7, 8]
    [13, 15, 18]
    [22, 24]
  [30, 40]
    [26, 27]
    [32, 35, 38]
    [42, 45, 46]
-----
```

Para a validação destes dados, foi utilizado o site “B-Trees”, indicado pelo docente em sala de aula, o que garantiu o resultado esperado.

3.3 Testes de Remoção

A operação de remoção foi testada para garantir que a árvore mantivesse a ordem e o balanceamento após a exclusão de chaves, a partir de um vetor definido na função *main* com chaves já existentes na árvore, como pode ser observado a seguir:

```
38 // Vetor de elementos a serem removidos da árvore
39 vector<int> elementos2 = {25, 45, 24};
```

Após cada remoção, o estado da árvore é impresso no terminal, onde os resultados obtidos foram:

```

-----
Remoção na Árvore B
-----

- Remove 25...

Arquivo btree.bin convertido para btree.txt com sucesso!
[24]
  [10, 18]
    [5, 7, 8]
    [13, 15]
    [20, 22]
  [30, 40]
    [26, 27]
    [32, 35, 38]
    [42, 45, 46]

-----

- Remove 45...

Arquivo btree.bin convertido para btree.txt com sucesso!
[24]
  [10, 18]
    [5, 7, 8]
    [13, 15]
    [20, 22]
  [30, 40]
    [26, 27]
    [32, 35, 38]
    [42, 46]

-----

- Remove 24...

Arquivo btree.bin convertido para btree.txt com sucesso!
[10, 22, 30, 40]
  [5, 7, 8]
  [13, 15, 18, 20]
  [26, 27]
  [32, 35, 38]
  [42, 46]
-----

```

Assim como na inserção de chaves, para a validação dos dados removidos, foi-se utilizado o site “B-Trees”, indicado pelo docente em sala de aula, o que garantiu o resultado esperado.

3.4 Testes de Busca

A operação de busca foi testada para avaliar a eficiência e correção da busca por uma chave na Árvore B, a partir de um vetor definido na função *main* com chaves já existentes na árvore, como pode ser observado a seguir:

```

53 // Vetor de de elementos a serem buscados na árvore. OBS.: A busca é feita no arquivo binário
54 vector<int> buscas = {3, 69, 15, 2};

```



```
-----
Busca na Árvore B
-----
- Chave 3 Não encontrada!
-----
- Chave 69 Não encontrada!
-----
Chave 15 encontrada no nó com endereço: 0x55a245d0cd10
-----
- Chave 2 Não encontrada!
-----
```

O resultado da busca ocorreu como esperado, podendo o resultado ser consultado no arquivo em formato texto.

4. Conclusão

Portanto, este trabalho teve como objetivo a implementação de uma Árvore B em C++, explorando operações tais como: inserção, remoção e busca, e permitindo o armazenamento da árvore em dois formatos: binário e texto. Embora o projeto em questão tenha sido bem-sucedido, houve desafios para garantir o balanceamento correto durante a operação de remoção, além de lidar com a manipulação de arquivos binários. Apesar disso, esses desafios foram superados por meio de depuração e da implementação de funções auxiliares que garantiram a integridade da árvore.

Nesse sentido, os testes revelaram que a Árvore B se comporta conforme esperado. Durante os testes de inserção, a árvore mantém o balanceamento e a ordem, com as chaves sendo distribuídas de forma adequada entre os nós, mesmo com uma grande quantidade de inserções. Nos testes de remoção, a árvore foi capaz de ajustar a estrutura corretamente, utilizando a divisão e o empréstimo de chaves, também mantendo a árvore balanceada. A busca por chaves também foi realizada com sucesso, demonstrando a eficiência da Árvore B.

Ademais, a funcionalidade de salvar e carregar a árvore a partir de arquivos binário e texto permitiu a continuidade das operações, garantindo que as modificações feitas em uma execução fossem preservadas nas próximas. Ainda, a busca direta no arquivo binário se mostrou uma maneira eficiente de acessar os dados sem precisar carregar na memória.

Com isso, o projeto alcançou seus objetivos, proporcionando uma boa compreensão acerca da implementação e funcionamento da Árvore B e de outras funcionalidades, tais como as técnicas de armazenamento de dados e manipulação de arquivos.